

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1

Software Design

Author: Andrei Radu Varga

Group: group 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

March 2026

Contents

1	Problem definition	2
2	Analysis phase - Use cases & requirements	2
2.1	Functional requirements	2
2.2	Use cases	2
3	Design phase	4
3.1	Architecture overview	4
3.2	Entity-relationship diagram	4
3.3	Presentation Component (Views)	5
3.4	Service Component	5
3.5	Design patterns	5
3.6	Class diagrams	6
3.7	Sequence diagrams	6
4	Implementation phase - Application	7
4.1	Tools and technologies	7
4.2	Application structure	7
4.3	User manual	8
5	Testing and validation	9

1 Problem definition

This project consists of developing an Online Clothes Marketplace desktop application based on the Model-View-Controller (MVC) architectural pattern. The application serves as a platform where users can browse different shops, view clothing products, place orders, and manage their activity depending on their access level.

The following sections describe the analysis, design, implementation, and testing phases of the application. The analysis phase details the system's functional requirements and use cases. The design phase illustrates the architecture, database schema, and design patterns. The implementation phase outlines the technologies used and the code structure, while the final section presents the testing strategy.

The deliverables for this assignment include the Java source code, the MySQL database schema, and this documentation detailing the software design process.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

Functional requirements describe the operations that the system must perform. These requirements define the services the system provides and the expected system behavior.

The core functional requirements for the Online Clothes Marketplace include:

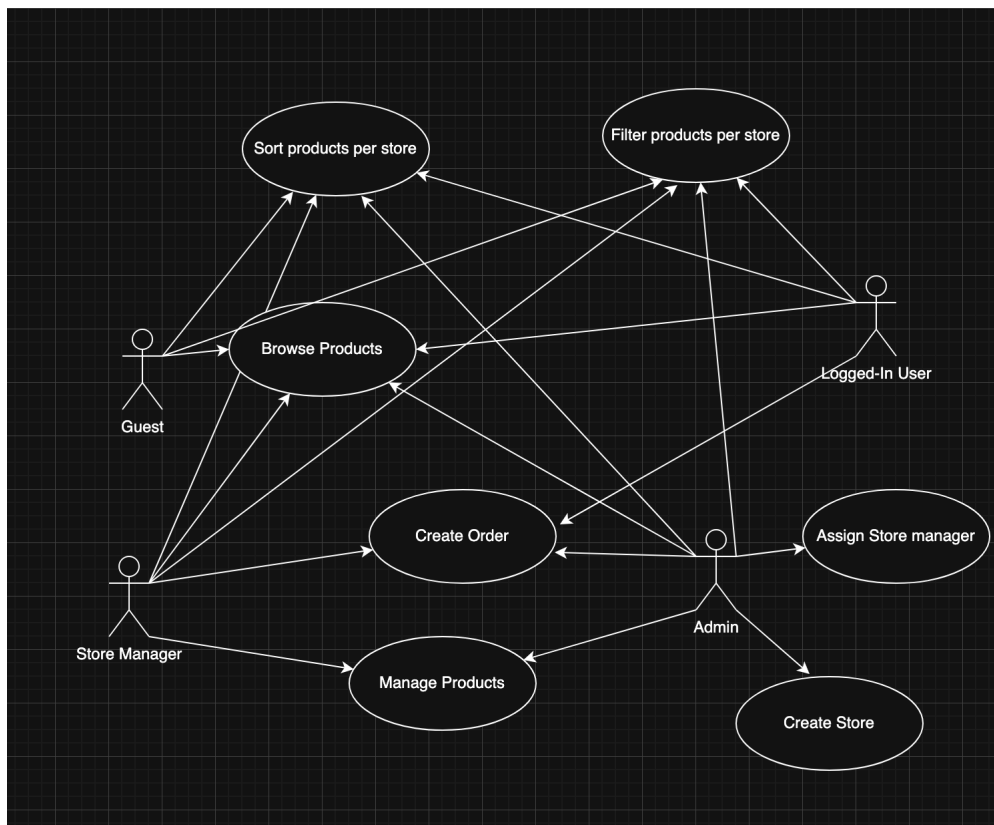
- The system must allow users to authenticate securely, while also supporting unauthenticated access.
- The system must allow guest users (not logged in) to browse available shops and view their clothing products.
- The system must allow registered customers to add products to an order and view their personal order history (My Orders).
- The system must allow store managers to create, read, update, and delete (CRUD) products exclusively within their assigned shop.
- The system must allow administrators to create, update, and delete all shop and product records across the platform.
- The system must allow administrators to manage user accounts and assign specific roles.

2.2 Use cases

Use cases describe in text the interactions between actors and the system in order to accomplish specific tasks. The primary actors in this system are the Guest, Customer, Store Manager, and Administrator.

- **Actor:** Guest (Not logged in)

- **Task:** Browse Catalog
- **Result:** The guest successfully views the list of available shops and browses their clothing items without the ability to place an order.
- **Actor:** Customer
- **Task:** Place Order
- **Result:** The customer adds items to their order and checks out. The system saves the new order in the database and updates their order history.
- **Actor:** Store Manager
- **Task:** Manage Shop Inventory
- **Result:** The store manager adds a new product or updates existing stock and prices for their specific shop. The system updates the catalog, making the changes visible to shoppers.
- **Actor:** Administrator
- **Task:** Manage Platform and Users
- **Result:** The administrator creates a new shop, assigns a user as a store manager, or modifies user roles.



3 Design phase

3.1 Architecture overview

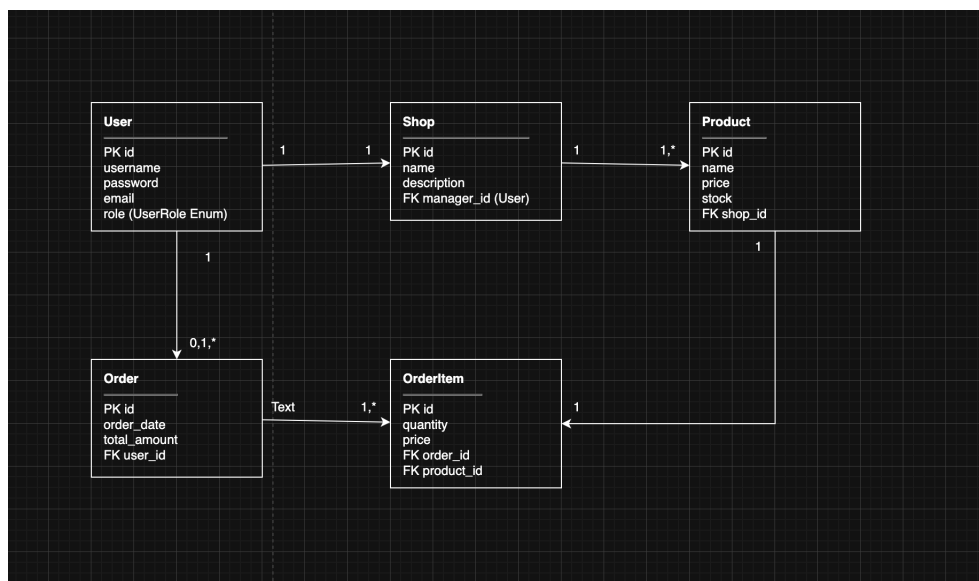
The application is built using a layered monolithic architecture following the Model-View-Controller (MVC) design pattern. The application is divided into presentation, business logic, and data access layers.

The presentation layer is built using Java Swing, providing a graphical user interface (GUI) for the user. The controller layer acts as an intermediary, capturing user inputs from the views and delegating them to the service layer. The service layer contains the core business logic, mapping Data Transfer Objects (DTOs) to entity models. Finally, the data access layer relies on Spring Data JPA to persist information in a relational MySQL database.

3.2 Entity-relationship diagram

The database schema represents the entities of the marketplace and their relations. The main tables include:

- **User:** Stores account credentials and personal details. It has a relationship with the UserRole table to manage access levels (Customer, Store Manager, Admin).
- **Shop:** Represents a clothing store on the marketplace. It has a one-to-many relationship with Products and can be linked to a specific Store Manager.
- **Product:** Stores clothing item details (name, price, stock) and belongs to a specific Shop.
- **Order:** Represents a purchase made by a logged-in User. It has a many-to-one relationship with the User table.
- **OrderItem:** Acts as a join entity between Order and Product, storing the quantity and price of specific clothing items purchased in a given order.



3.3 Presentation Component (Views)

The View component comprises Java Swing components, heavily utilizing `JPanel` and `JFrame` for classes such as `BrowseShopsPanel`, `ManageProductsPanel`, and `MyOrdersPanel`. This component is strictly responsible for rendering the UI and capturing user events (such as button clicks or text field inputs). It forwards these actions to the appropriate controllers via action listeners without containing any business logic itself.

3.4 Service Component

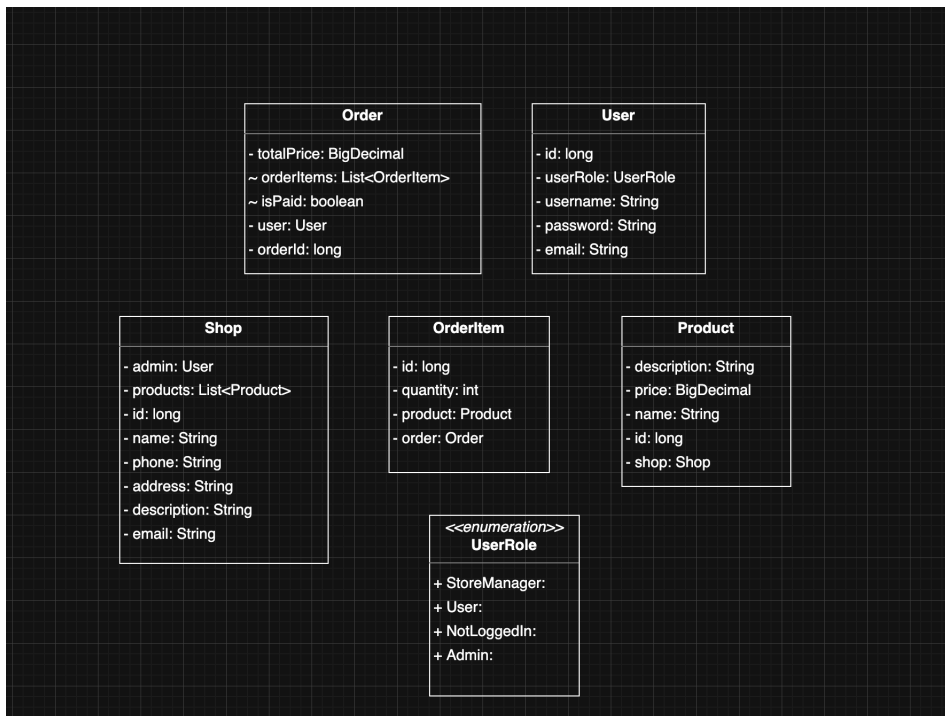
The Service layer (e.g., `ProductServiceImpl`, `OrderServiceImpl`) bridges the Controllers and the Repositories. It is responsible for enforcing business rules, performing calculations, and utilizing Mapper classes (`ProductMapper`, `UserMapper`) to convert between internal Models and DTOs (Data Transfer Objects) to ensure that the database schema is completely decoupled from the UI layer.

3.5 Design patterns

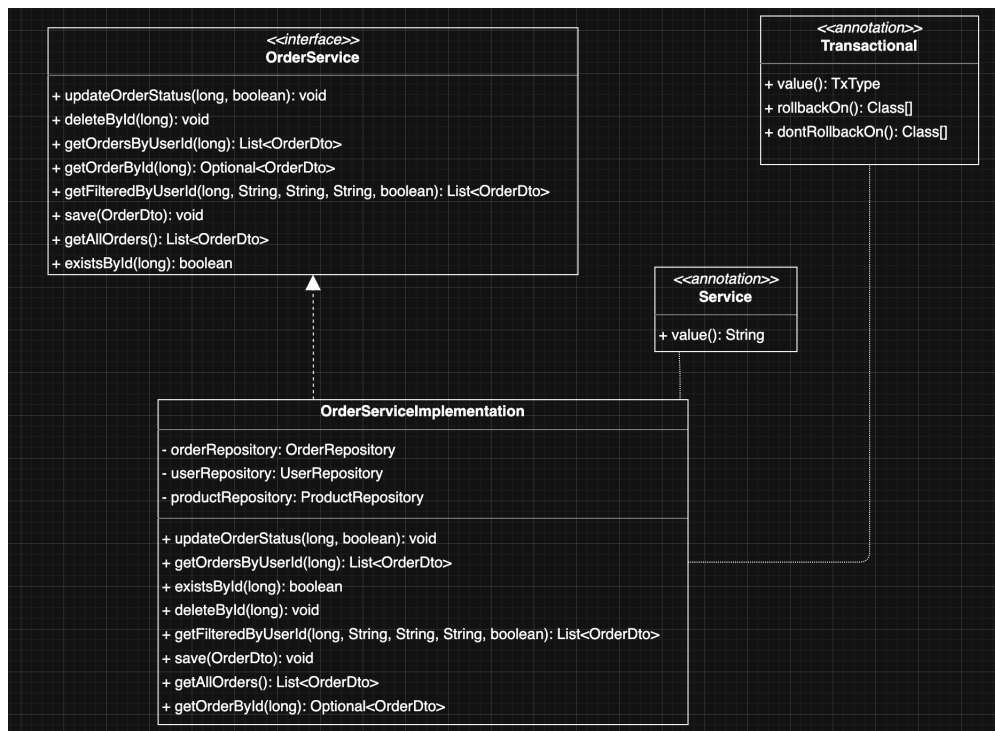
The system employs several design patterns to ensure clean code and maintainability:

- **Model-View-Controller (Architectural):** Separates the application into three interconnected components, separating internal representations of information from the ways information is presented to and accepted from the user.
- **Builder Pattern (Creational):** Used extensively via Lombok (e.g., `UserBuilder`, `ProductBuilder`, `ShopBuilder`) to facilitate the step-by-step creation of complex data objects without cluttering constructors.
- **Data Transfer Object (Structural):** Used to encapsulate data (`ShopDto`, `UserDto`, etc.) and send it from the service layer to the client/view layer, limiting the exposure of database entities.
- **Singleton Pattern (Creational):** Managed via Spring Boot's dependency injection to ensure only one instance of services and repositories are created and shared across the application.

3.6 Class diagrams



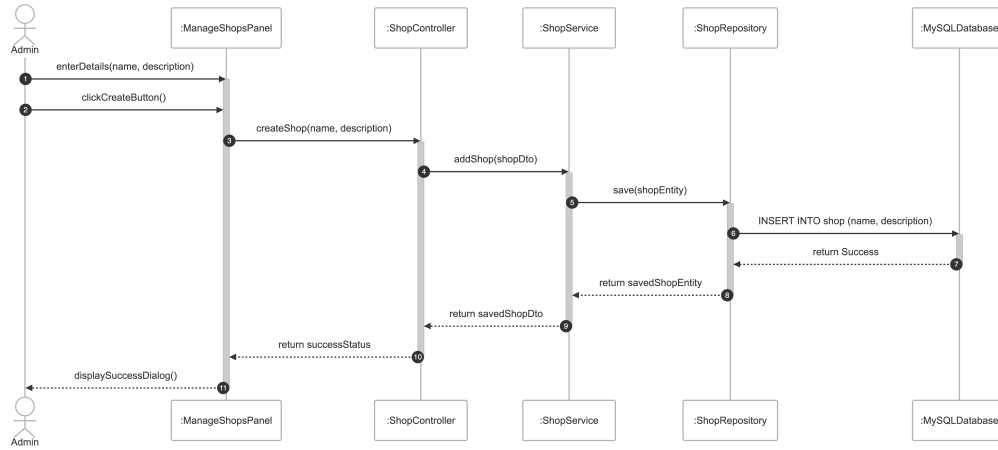
Model Classes



Example of Service Class

3.7 Sequence diagrams

Sequence diagrams illustrate how objects interact during the execution of a particular use case.



Sequence diagram for shop creation for Admin

4 Implementation phase - Application

4.1 Tools and technologies

The application is built using a modern Java ecosystem. The main technologies include:

- **Java 17:** The core programming language used for development.
- **Java Swing:** The standard Java GUI toolkit used to build the desktop presentation layer.
- **Spring Boot 4.0.3:** Used as the foundational framework to manage dependencies, application context, and configurations.
- **MySQL & Hibernate (Spring Data JPA):** MySQL serves as the relational database, while Hibernate is used as the Object-Relational Mapping tool to automatically generate database tables (via `ddl-auto=update`) and perform CRUD operations.
- **Maven:** Used for dependency management and project building.
- **Lombok:** A library used to reduce boilerplate code, automatically generating getters, setters, and builders for entities and DTOs.

4.2 Application structure

The source code is organized into distinct packages representing the architectural layers:

- **Controller:** Receives input from the Views and triggers Service methods.
- **Model & Dto:** Contains the JPA database entities and Data Transfer Objects.
- **View:** Contains the Swing GUI classes and UI panels.
- **Service:** Contains interfaces and implementations for the business logic, alongside mapping utilities.
- **Repositories:** Contains Spring Data JPA interfaces for database access.
- **Security:** Contains utility classes like `UserSession` to track logged-in users and manage their access rights based on their role.

4.3 User manual

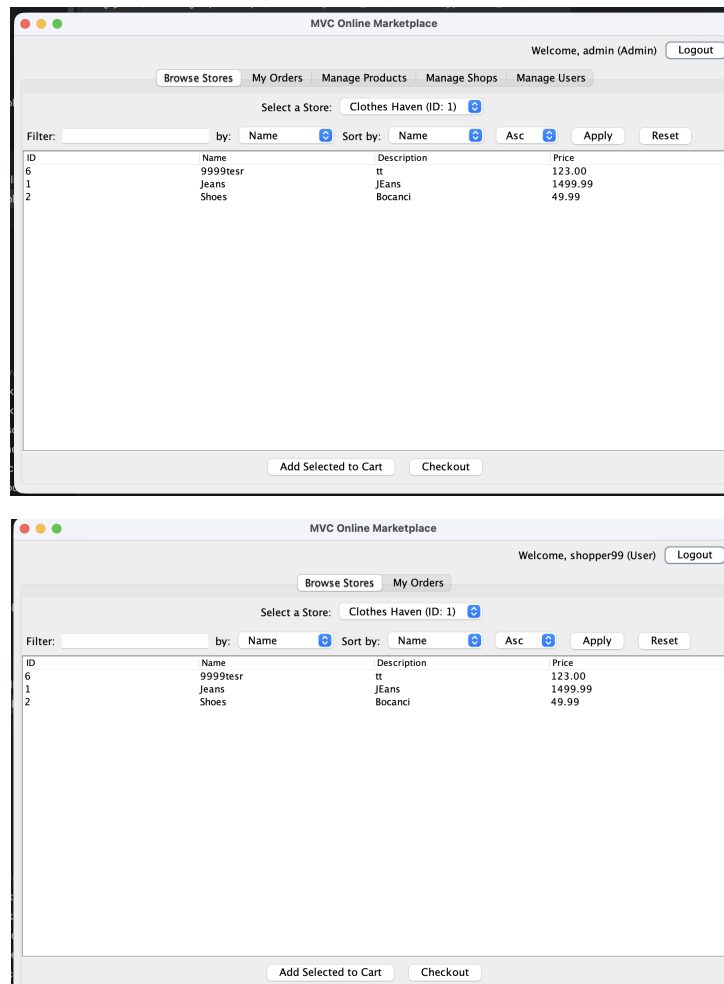
Upon launching the MVCOnlineMarketplace application, users interact with the MainFrame. Depending on their authentication status and UserRole, they have access to different functionalities:

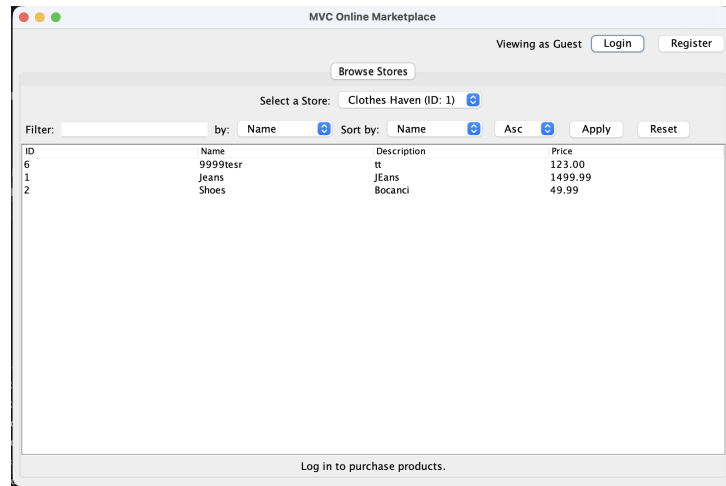
Guests (Not logged in) can launch the application and navigate the BrowseShopsPanel to explore the available stores and view clothing products, but cannot place any orders.

Customers log in to browse products and can use the ordering system to purchase items. They also have access to the MyOrdersPanel to check their purchase history.

Store Managers log in and gain access to the ManageProductsPanel for their designated shop, allowing them to add, update, or remove inventory for their specific catalog.

Administrators have complete oversight and access to management panels, such as the ManageUsersPanel, ManageShopsPanel, and global ManageProductsPanel, allowing them to maintain the entire marketplace ecosystem.





5 Testing and validation

This section describes the methods used to verify that the application works correctly.

Testing was done through the GUI.